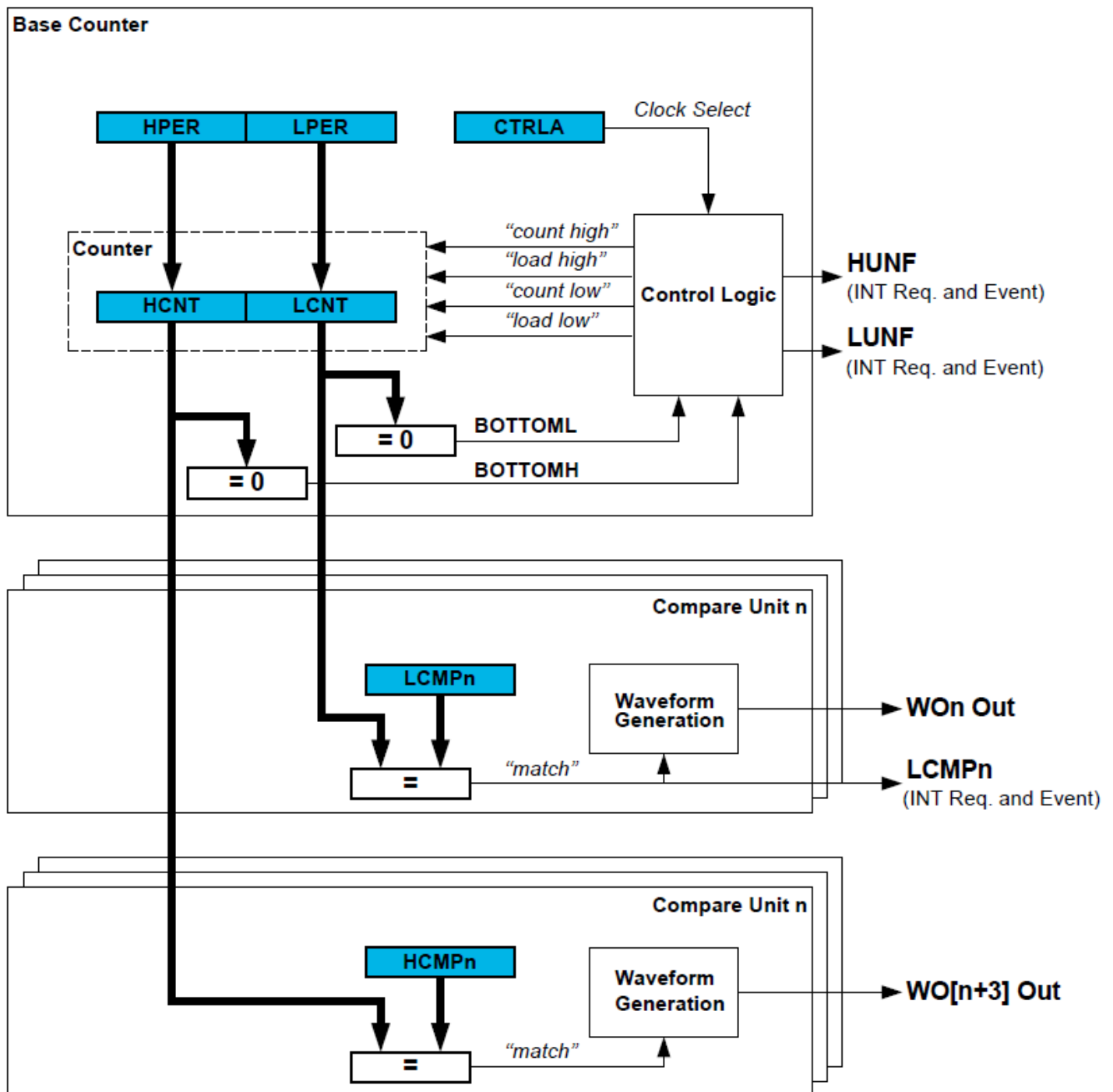


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 8 - TCA Split Mode



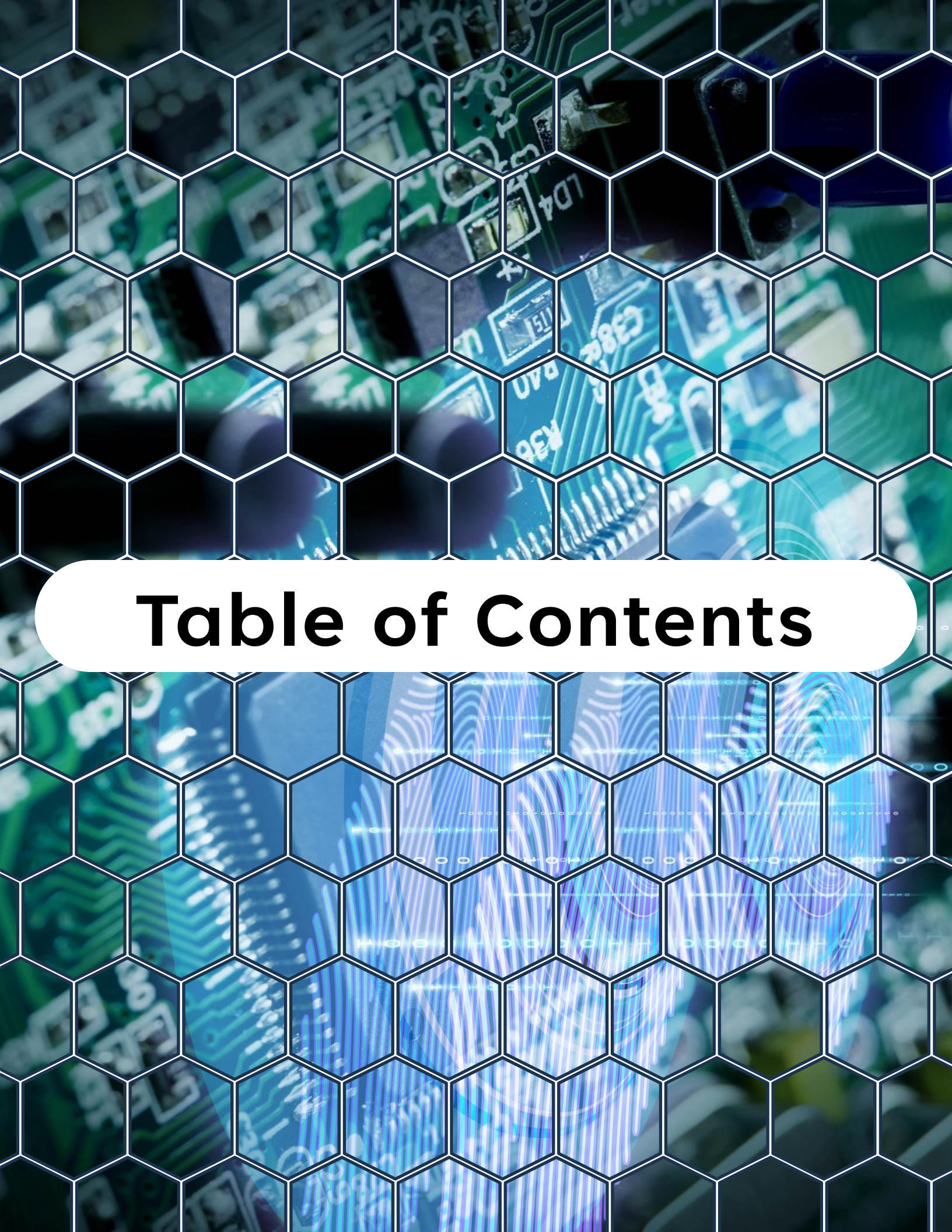
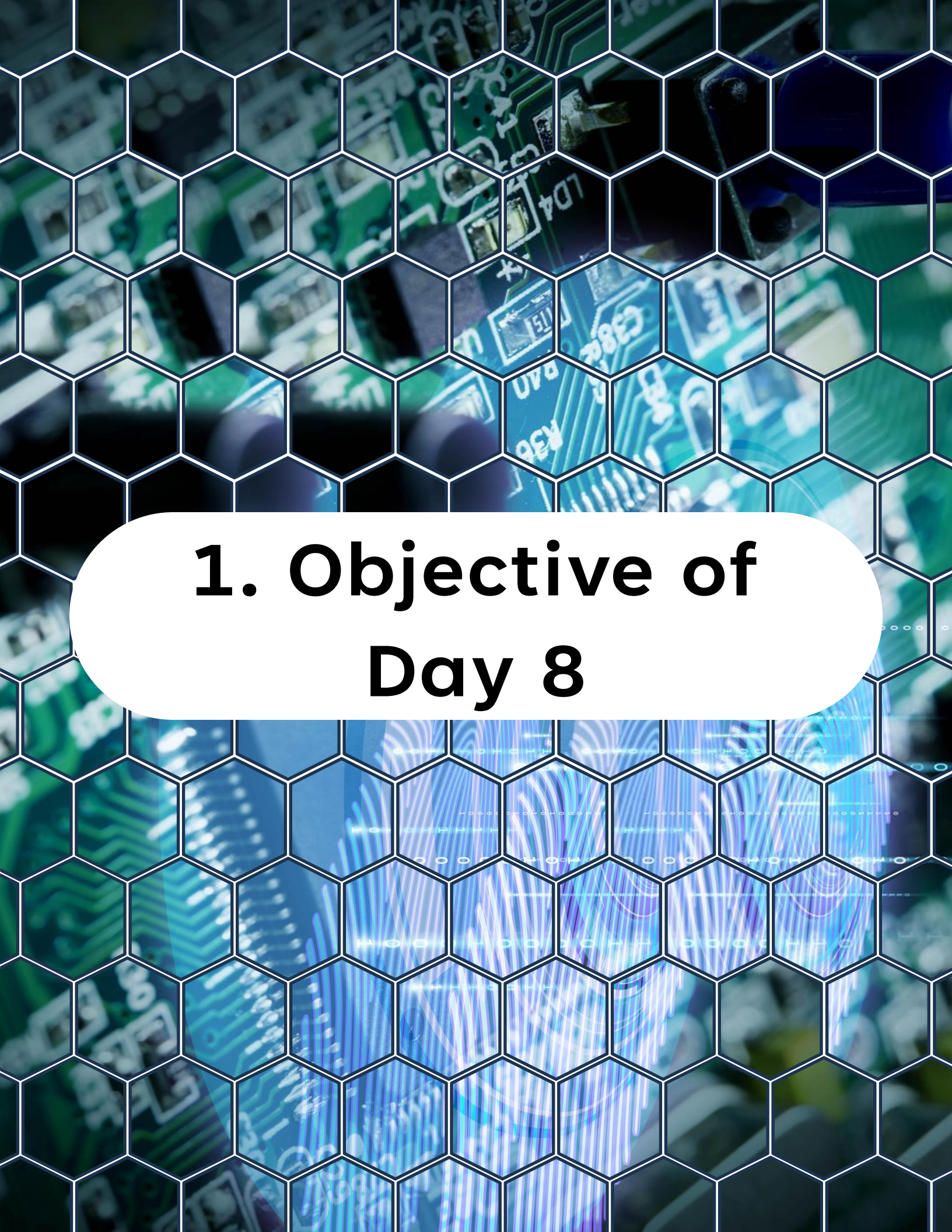


Table of Contents

Table of Contents

1. Objective of Day 8
2. Why Split Mode Matters
3. Split Mode Concept (How It Works)
4. What You Get in Split Mode
5. Split Mode Tradeoffs
6. Clocking and Frequency Math
7. Duty Cycle Math
8. Output Routing (PORTMUX Reminder)
9. Practical Lab - Two PWM Frequencies, Multiple Outputs
10. Code Example - TCA0 Split Mode PWM (4 Channels)
11. Updating Duty in Split Mode
12. Minimal main() Example
13. Debugging Split Mode Problems
14. What You Learned Today
15. What Comes Next



1. Objective of Day 8

1. Objective of Day 8

Today you will configure TCA in Split mode.

Split mode turns one 16-bit TCA timer into two independent 8-bit timers:

- A Low (L) timer
- A High (H) timer

Each side can generate its own PWM signals.

This is super useful when you want:

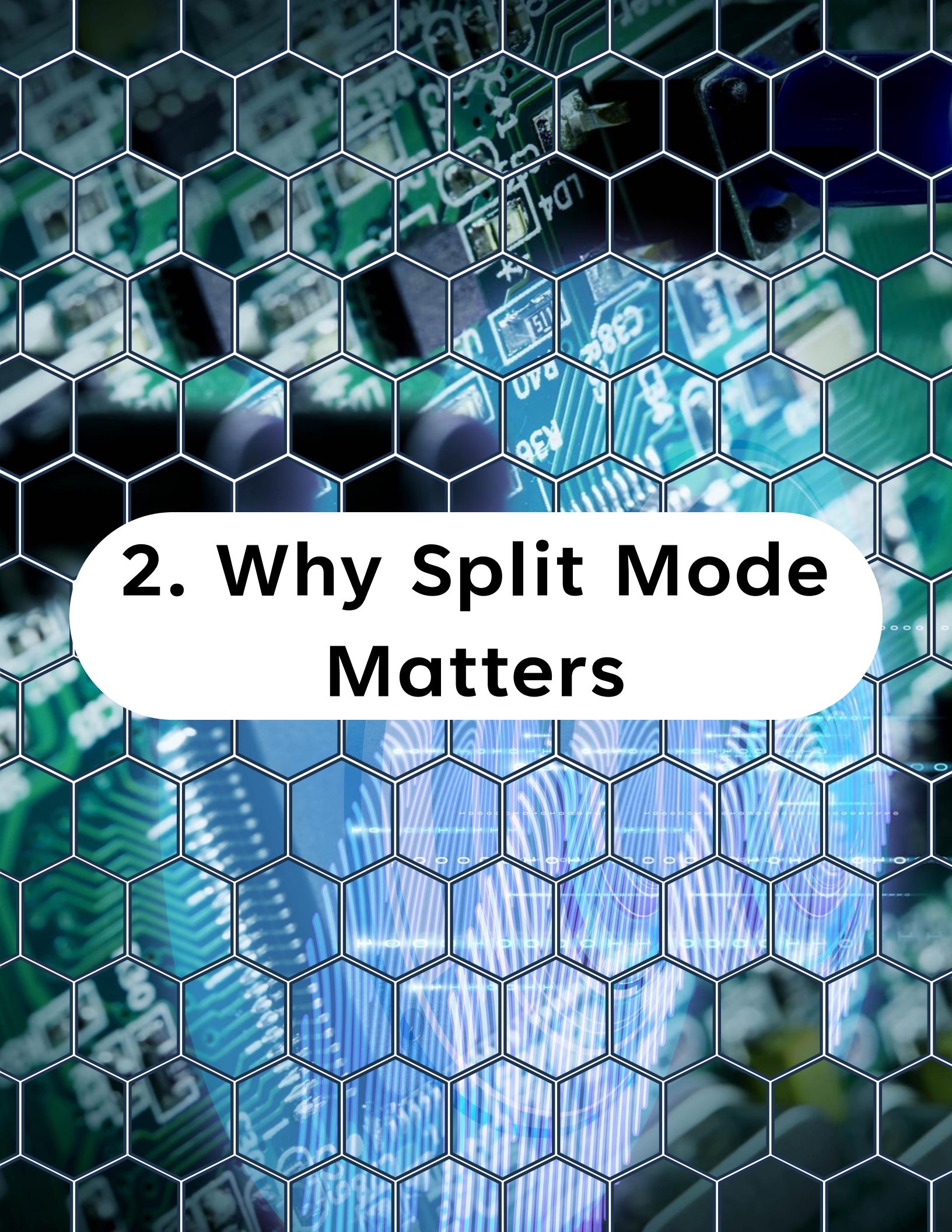
- More PWM channels
- Simple 8-bit duty control
- Independent PWM periods for different loads

By the end of this day, you will:

- Understand what TCA Split mode is and why it exists

1. Objective of Day 8

- Configure Split mode at register level
- Generate PWM on multiple waveform outputs
- Set independent periods and duty cycles for the L and H sides
- Know the tradeoffs vs single mode



2. Why Split Mode Matters

2. Why Split Mode Matters

TCA Single mode is great when:

- You want high resolution (16-bit period)
- You want one shared PWM frequency across channels

But real designs often need:

- Many PWM channels
- Different PWM frequencies for different subsystems
- Good-enough 8-bit resolution (0 to 255 duty)

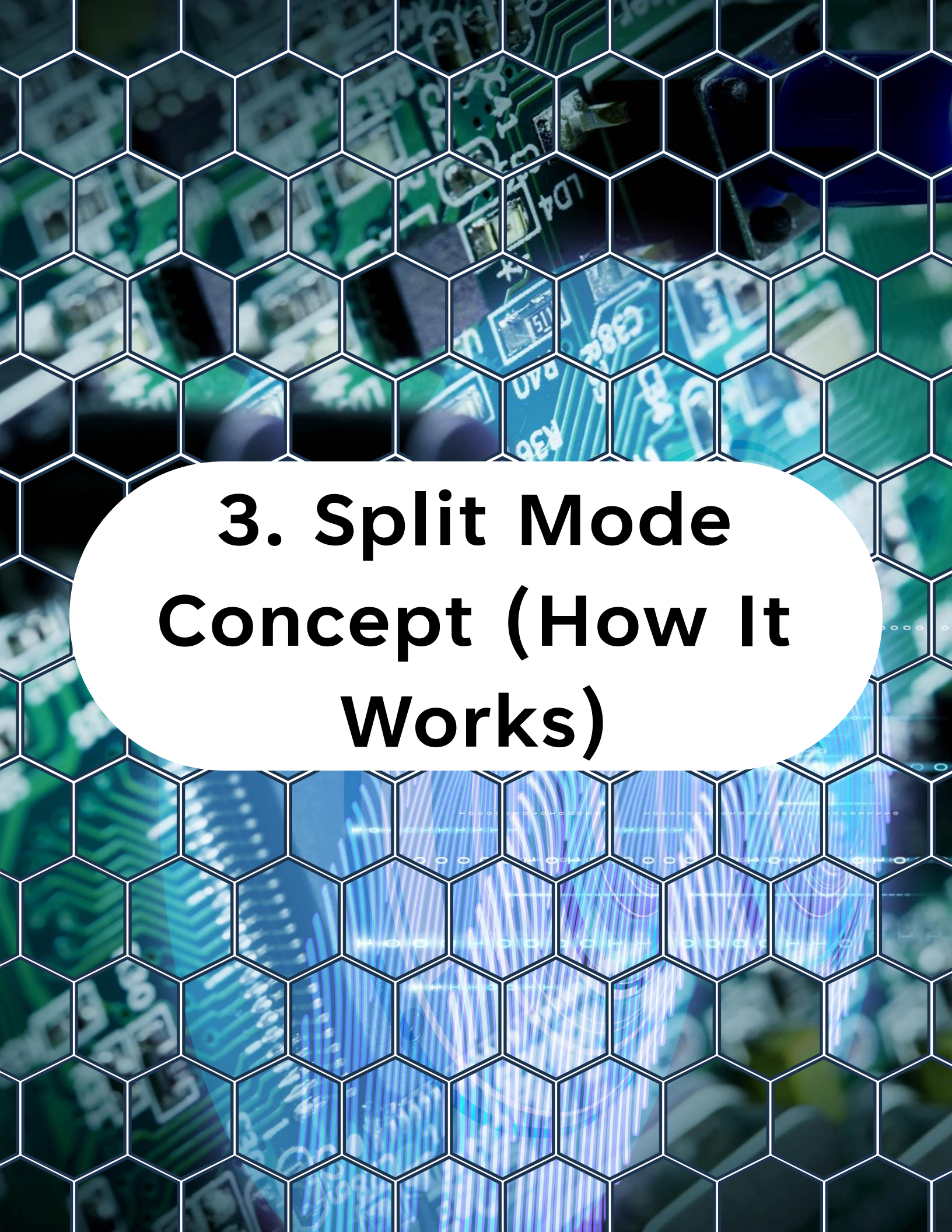
Split mode is designed for exactly that.

This is common in:

- Multi-channel LED control
- Mixed loads (fan PWM + LED PWM)

2. Why Split Mode Matters

- Power electronics where different signals need different base frequencies
- Building blocks for multi-output control systems



3. Split Mode Concept (How It Works)

3. Split Mode Concept (How It Works)

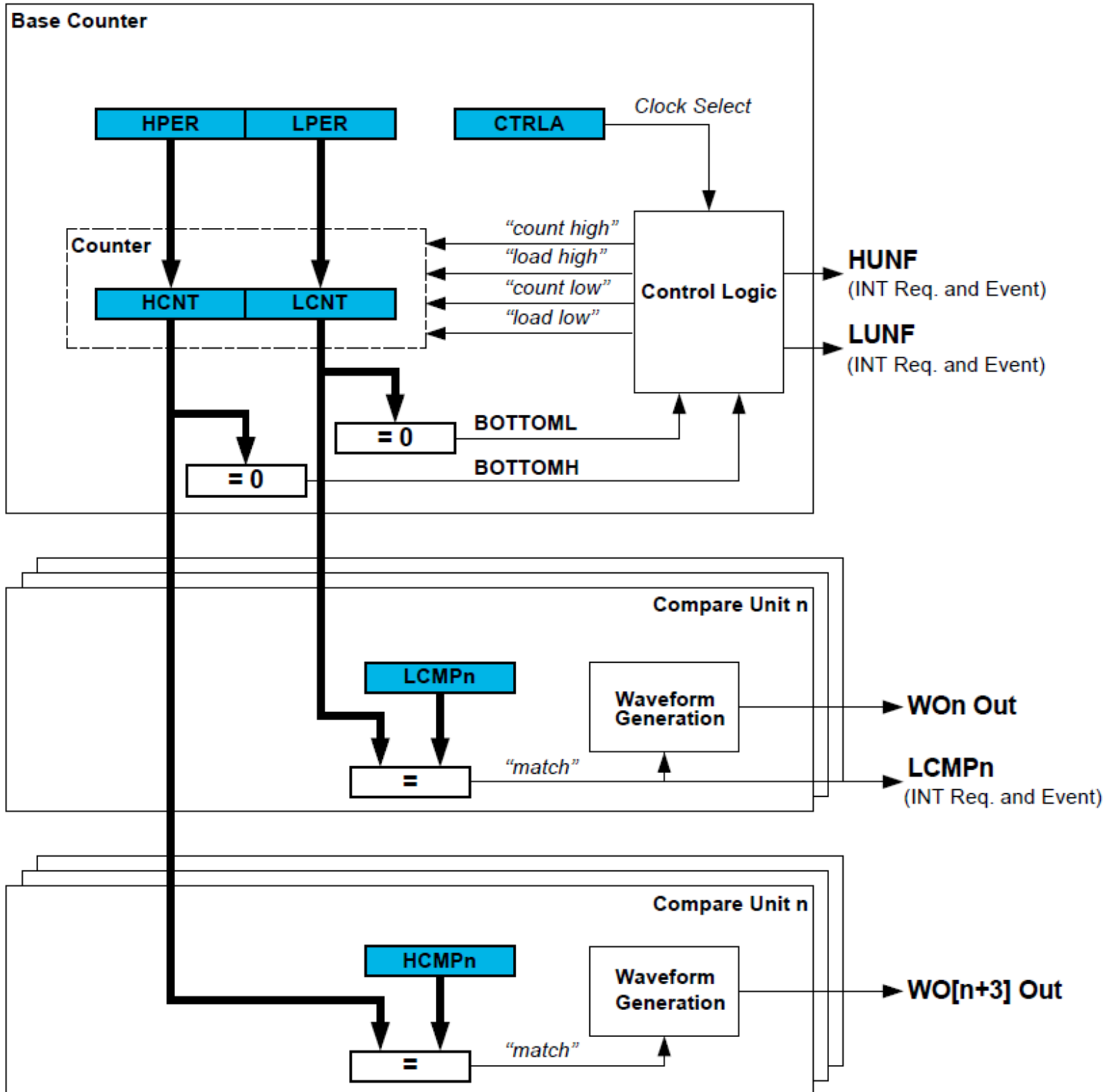
In Split mode:

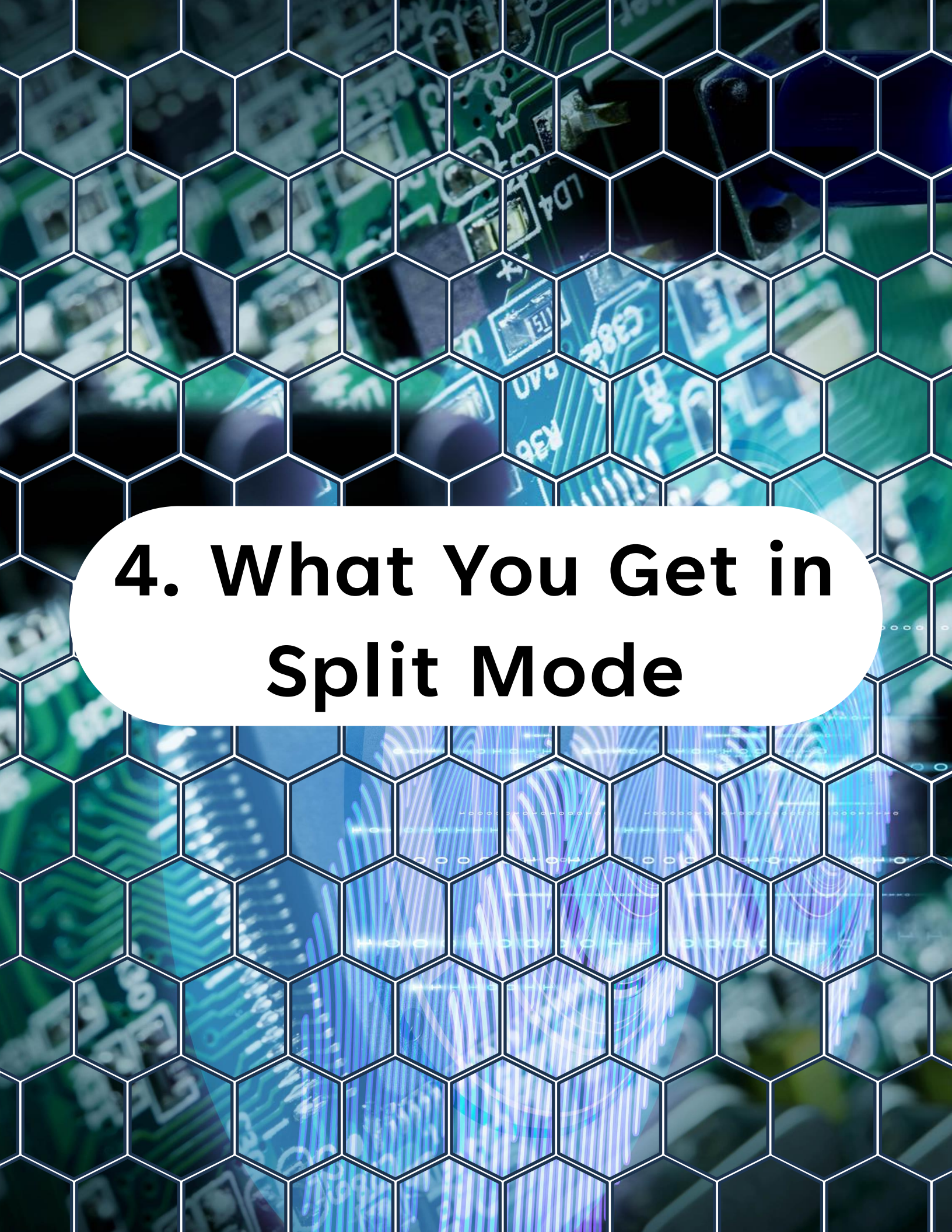
- The 16-bit counter becomes two 8-bit counters
- Each half has its own period register
- Each half has its own compare registers (three compare channels for PWM generation)
- Each half drives its own waveform outputs
- Split mode will only work with single-slope down-count

You still have one TCA peripheral, but it behaves like two smaller timers.

3. Split Mode Concept (How It Works)

Timer/Counter Block Diagram Split Mode





4. What You Get in Split Mode

4. What You Get in Split Mode

In general terms, you get:

One Low timer with:

- **LPER** (period)
- **LCMP0/1/2** (compare)
- Outputs for **WO0**, **WO1**, **WO2** (depending on routing)

One High timer with:

- **HPER** (period)
- **HCMP0/1/2** (compare)
- Outputs for **WO3**, **WO4**, **WO5** (depending on routing)

So you can drive up to **6 PWM** outputs total.

4. What You Get in Split Mode

Exact pin mapping depends on:

- Your device
- Your board routing
- PORTMUX selection



5. Split Mode Tradeoffs

5. Split Mode Tradeoffs

Split mode gives flexibility, but you pay for it:

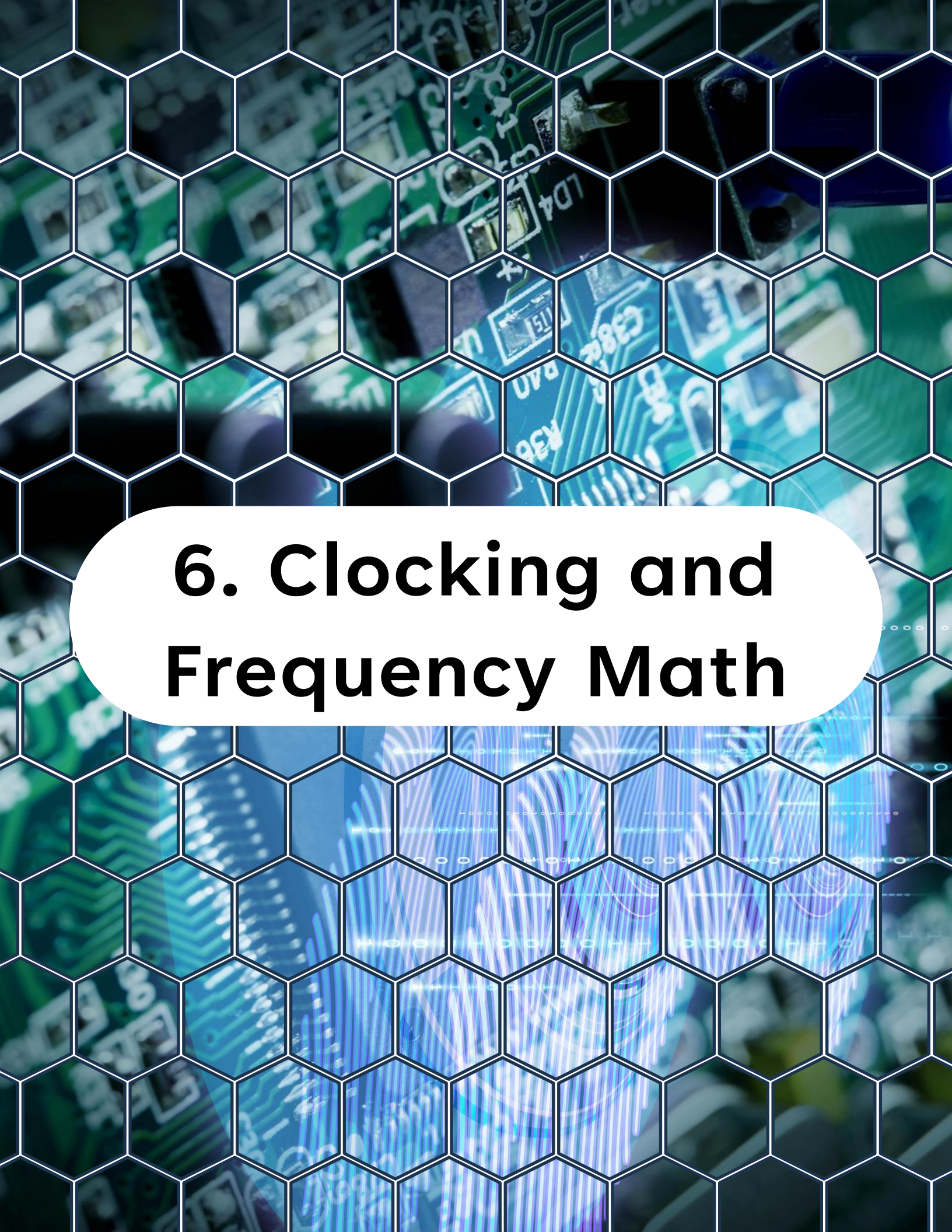
Pros:

- More PWM channels
- Two independent periods (two frequencies)
- Simple 8-bit duty math

Cons:

- Only 8-bit resolution for period and duty
- Not suitable for low-frequency PWM that needs large period counts
- Some advanced single-mode features do not apply the same way

This is a common embedded trade: resolution vs channel count.



6. Clocking and Frequency Math

6. Clocking and Frequency Math

In Split mode, each side (L and H) is 8-bit.

A practical formula:

$$\text{PWM_freq} = \text{F_tca} / (\text{PER} + 1)$$

Where:

F_tca is the TCA clock after prescaler

PER is LPER or HPER

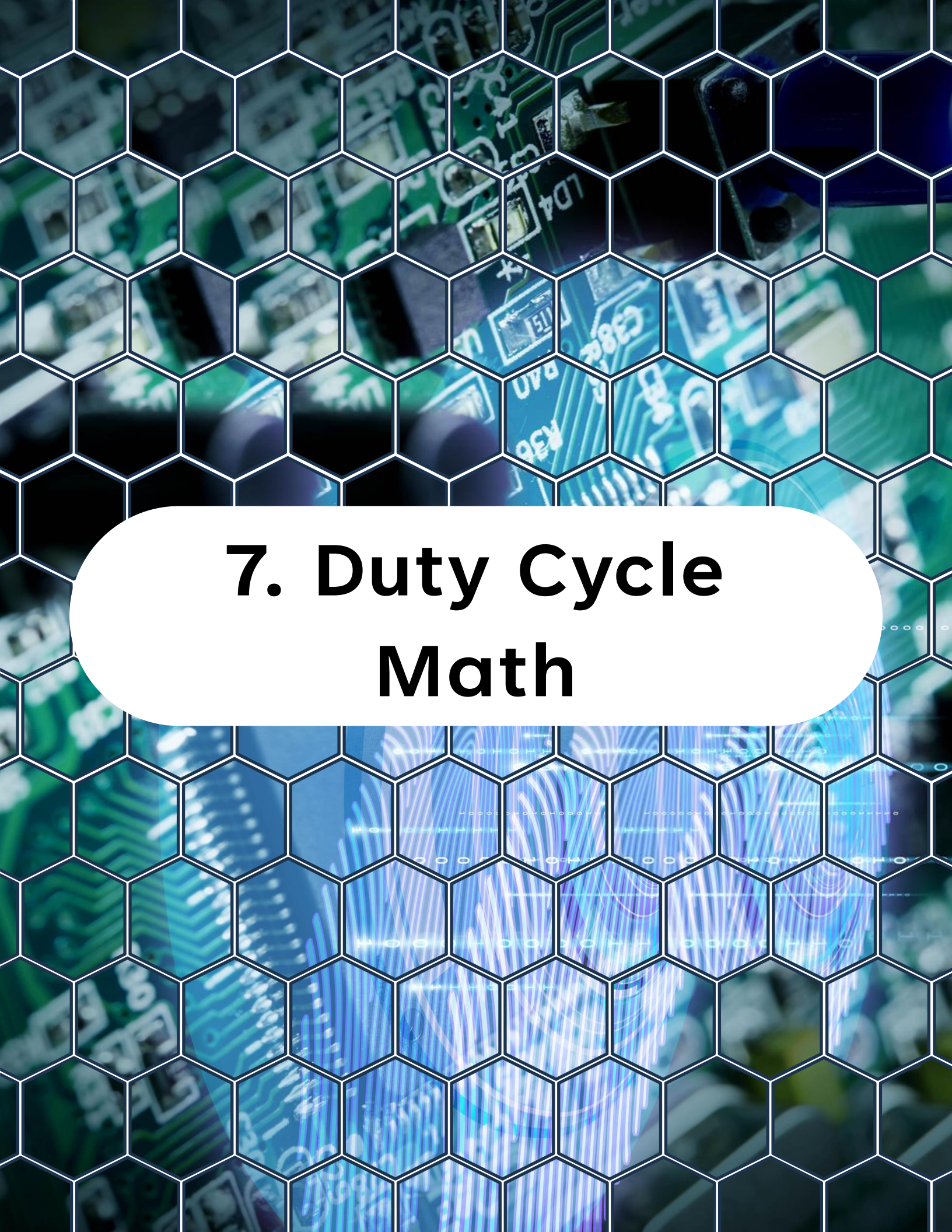
Example:

$$\text{F_tca} = 24 \text{ MHz} / 64 = 375 \text{ kHz}$$

$$\text{PER} = 249$$

$$\text{PWM} = 375 \text{ kHz} / 250 = 1500 \text{ Hz}$$

The range is limited because PER is only 0..255.



7. Duty Cycle Math

7. Duty Cycle Math

Duty is set by compare registers:

$$\text{Duty} = \text{CMP} / (\text{PER} + 1)$$

Because CMP is also 8-bit, duty is basically:

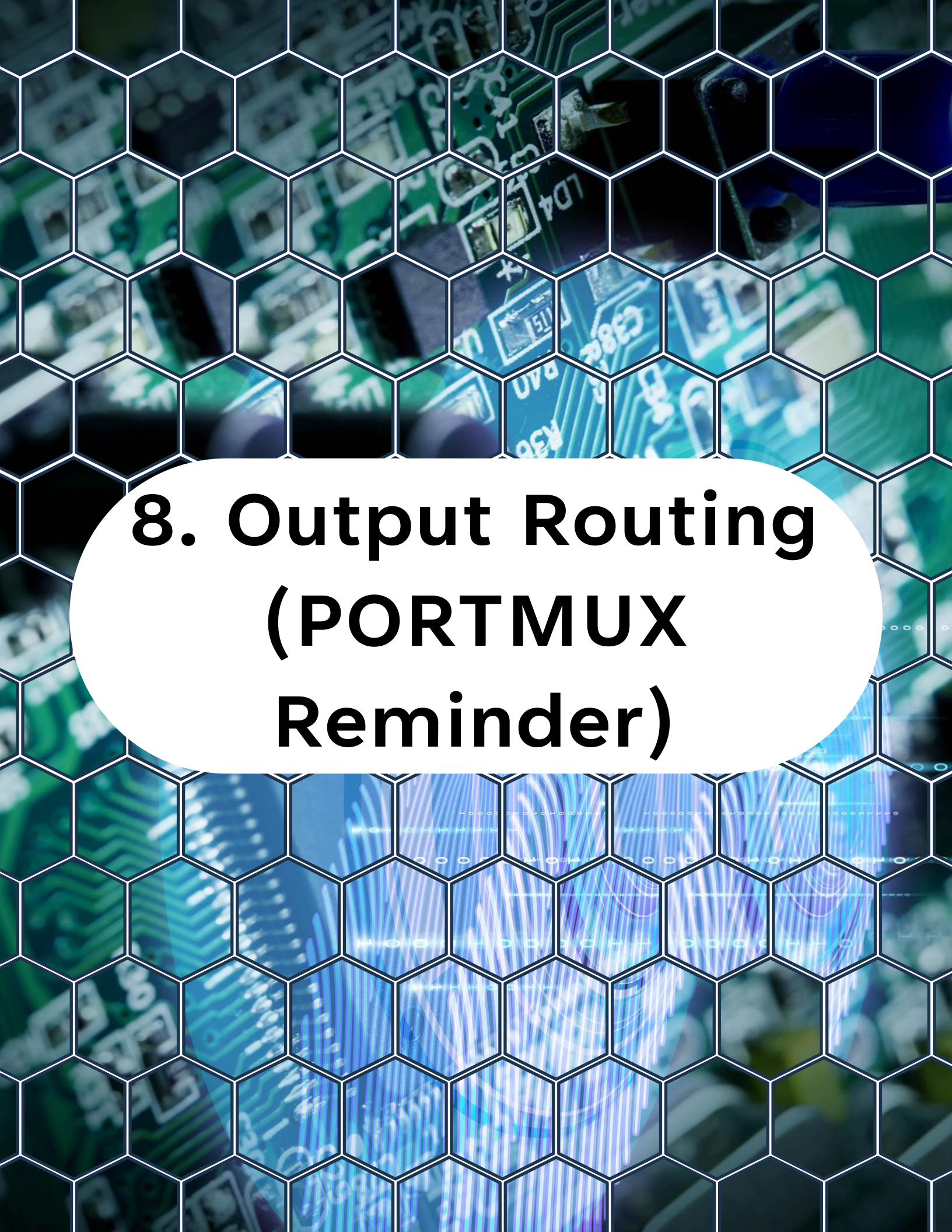
- 0..PER

In practice:

- $\text{CMP} = 0 \rightarrow$ nearly always low
- $\text{CMP} = \text{PER} \rightarrow$ nearly always high



```
1 static void tca0_pwm_set_duty_percent(uint8_t duty_percent)
2 {
3     uint32_t top = (uint32_t)TCA0.SINGLE.PER + 1UL;
4     uint32_t cmp = (top * (uint32_t)duty_percent) / 100UL;
5
6     if (cmp > 0)
7     {
8         cmp -= 1; /* keep within range for edge cases */
9     }
10
11     TCA0.SINGLE.CMP0 = (uint16_t)cmp;
12 }
```

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K30'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in a bold, black, sans-serif font.

8. Output Routing (PORTMUX Reminder)

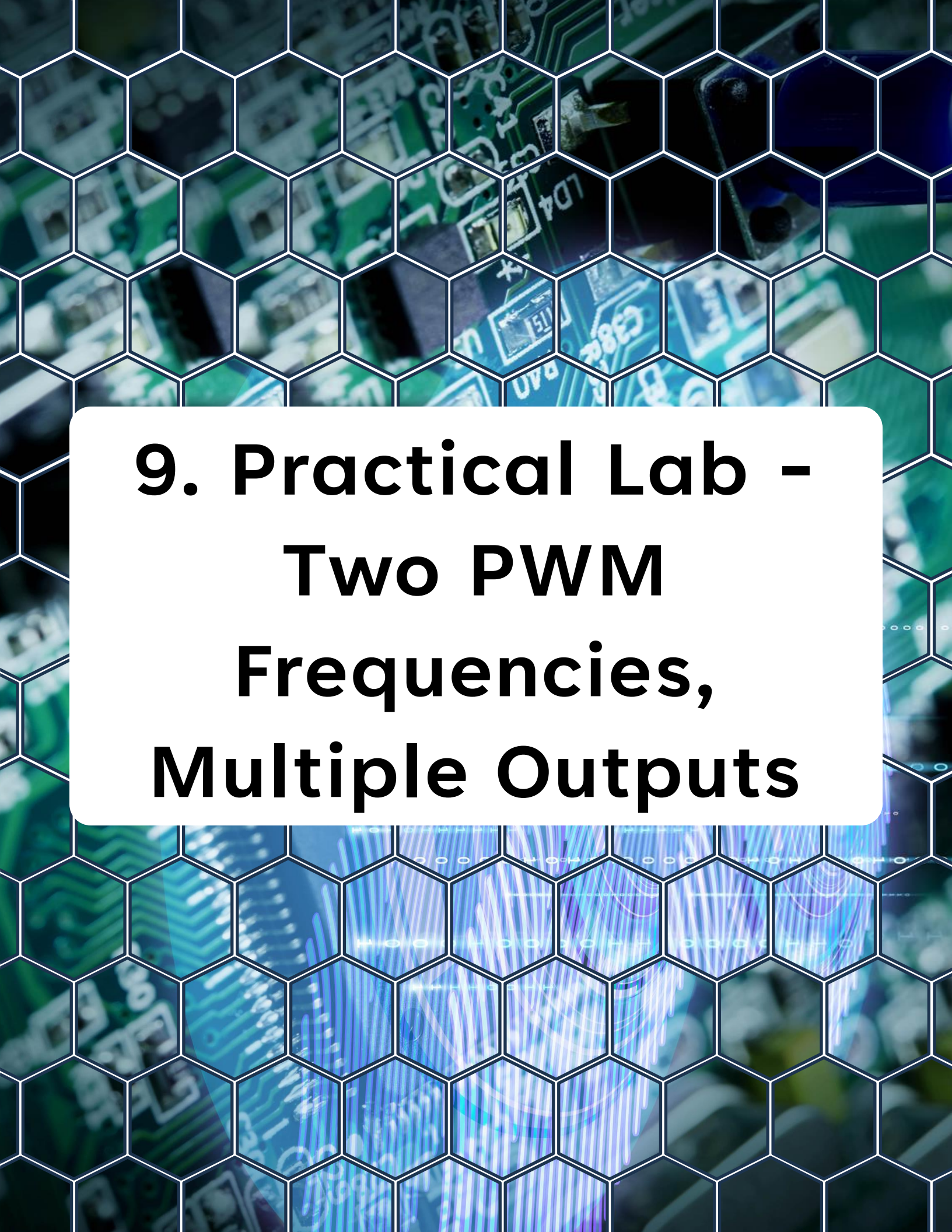
8. Output Routing (PORTMUX Reminder)

Just like Day 7, PWM does not appear on a pin unless:

- The **WO** pin is correctly routed (**PORTMUX**)
- The pin **DIR** is set to output

If you configure Split mode perfectly but forget **PORTMUX** or **DIR**, you will see nothing.

```
1  /* TCA0 Signals select */
2  typedef enum PORTMUX_TCA0_enum
3  {
4      PORTMUX_TCA0_PORTA_gc = (0x00<<0), /* PA0, PA1, PA2, PA3, PA4, PA5 */
5      PORTMUX_TCA0_PORTB_gc = (0x01<<0), /* PB0, PB1, PB2, PB3, PB4, PB5 */
6      PORTMUX_TCA0_PORTC_gc = (0x02<<0), /* PC0, PC1, PC2, PC3, PC4, PC5 */
7      PORTMUX_TCA0_PORTD_gc = (0x03<<0), /* PD0, PD1, PD2, PD3, PD4, PD5 */
8      PORTMUX_TCA0_PORTE_gc = (0x04<<0), /* PE0, PE1, PE2, PE3, PE4, PE5 */
9      PORTMUX_TCA0_PORTF_gc = (0x05<<0) /* PF0, PF1, PF2, PF3, PF4, PF5 */
10 } PORTMUX_TCA0_t;
11
12 // Route TCA0 outputs to PORTA
13 PORTMUX.TCAROUTEA = PORTMUX_TCA0_PORTA_gc;
14
15 // WO0 pin (PA0) as output
16 PORTA.DIRSET = PIN0_bm;
```

9. Practical Lab - Two PWM Frequencies, Multiple Outputs

9. Practical Lab - Two PWM Frequencies, Multiple Outputs

Lab Goal

- Configure Split mode
- Generate PWM on:
 - Low side: **WO0** and **WO1** (same **LPER**, different duty)
 - High side: **WO3** and **WO4** (same **HPER**, different duty)
- Use different periods on L and H sides so you can see different frequencies

This proves Split mode is truly two independent timers.

10. Code
Example - TCA0
Split Mode PWM
(4 Channels)

10. Code Example - TCA0 Split Mode PWM (4 Channels)

Important:

- The **WO** pin locations are board-dependent.
- The example assumes outputs map to **PORTA** pins for demonstration.
- Adjust pin selection based on your Curiosity Nano pinout and **PORTMUX** option you choose.

```
1  #include <avr/io.h>
2  #include <util/delay.h>
3
4  /**
5   * Initializes TCA0 in Split mode.
6   */
7  static void tca0_split_pwm_init(void)
8  {
9      // Stop TCA0 before configuration
10     TCA0.SINGLE.CTRLA = 0;
11
12     // Set the pin positions for TCA0 signals (PA0, PA1, PA2, PA3, PA4, PA5)
13     PORTMUX.TCAROUTEA = PORTMUX_TCA0_PORTA_gc;
14
15     // Configure output pins as outputs (PA0, PA1, PA2, PA3)
16     PORTA.DIRSET = PIN0_bm | PIN1_bm | PIN3_bm | PIN4_bm;
17
18     // Enable Split mode
```

10. Code Example - TCA0 Split Mode PWM (4 Channels)

```
18 // Enable Split mode
19 TCA0.SPLIT.CTRLD = TCA_SPLIT_SPLITM_bm;
20
21 /*
22  * Select a prescaler for the TCA clock.
23  * This affects both L and H timers.
24  */
25 TCA0.SPLIT.CTRLA = TCA_SPLIT_CLKSEL_DIV64_gc;
26
27 /*
28  * Set independent periods for L and H.
29  * LPER and HPER are 8-bit (0..255).
30  */
31 TCA0.SPLIT.LPER = 199; // Low side PWM period
32 TCA0.SPLIT.HPER = 99; // High side PWM period (different freq)
33
34 /*
35  * Set compare values (duty control).
36  * LCMPx -> affects W00/W01/W02 (low side)
37  * HCMPx -> affects W03/W04/W05 (high side)
38  */
39 TCA0.SPLIT.LCMP0 = 50; // W00 duty
40 TCA0.SPLIT.LCMP1 = 150; // W01 duty
41
42 TCA0.SPLIT.HCMP0 = 20; // W03 duty
43 TCA0.SPLIT.HCMP1 = 70; // W04 duty
44
45 /*
46  * Enable waveform outputs.
47  * In split mode, you enable the W0 channels you need.
48  */
49 TCA0.SPLIT.CTRLB = TCA_SPLIT_LCMP0EN_bm
50                  | TCA_SPLIT_LCMP1EN_bm
51                  | TCA_SPLIT_HCMP0EN_bm
52                  | TCA_SPLIT_HCMP1EN_bm;
53
54 // Enable TCA0, and start the timer
55 TCA0.SPLIT.CTRLA |= TCA_SPLIT_ENABLE_bm;
56 }
```

10. Code Example - TCA0 Split Mode PWM (4 Channels)

What you should see:

- **WO0** and **WO1** share the same frequency (**LPER**) but different duty
- **WO3** and **WO4** share the same frequency (**HPER**) but different duty
- L and H frequencies differ (because **LPER** != **HPER**)

This is the core proof that Split mode works.



11. Updating Duty in Split Mode

11. Updating Duty in Split Mode

In Split mode, duty updates are straightforward: just write LCMPx or HCMPx. This helper sets duty in percent for the Low side channel 0.

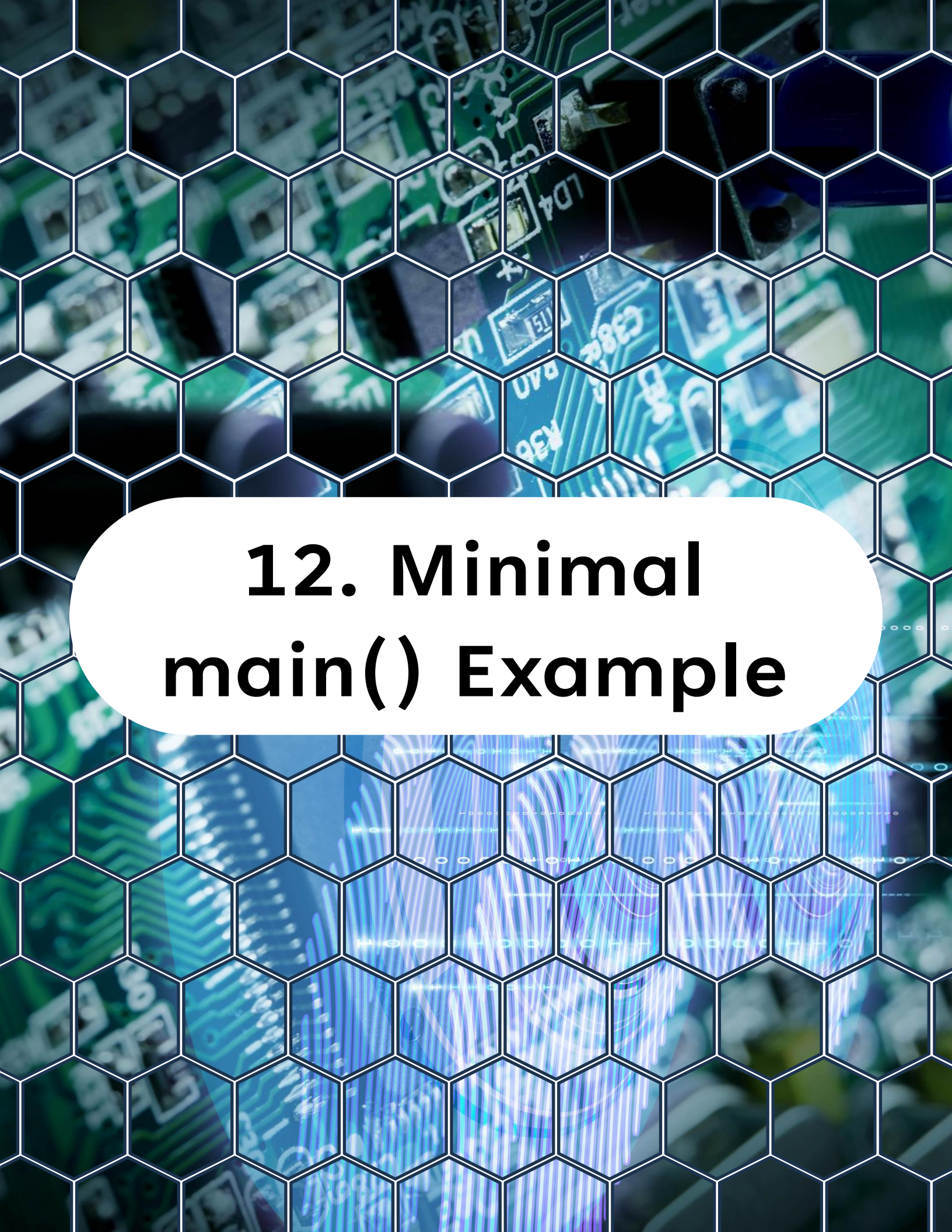
```
1  /**
2   * Helper to sets duty cycle in percent (0 to 100) for the
3   * Lower Compare Channel 0.
4   */
5  static void tca0_split_set_lcmp0_duty_percent(uint8_t duty_percent)
6  {
7      uint16_t top = (uint16_t)TCA0.SPLIT.LPER + 1U;
8      uint16_t cmp = (top * (uint16_t)duty_percent) / 100U;
9
10     if (cmp > 0)
11     {
12         cmp -= 1;
13     }
14
15     TCA0.SPLIT.LCMP0 = (uint8_t)cmp;
16 }
```

11. Updating Duty in Split Mode

Make similar helpers for:

- LCMP1, LCMP2
- HCMP0, HCMP1, HCMP2

Keep these helpers small and predictable.



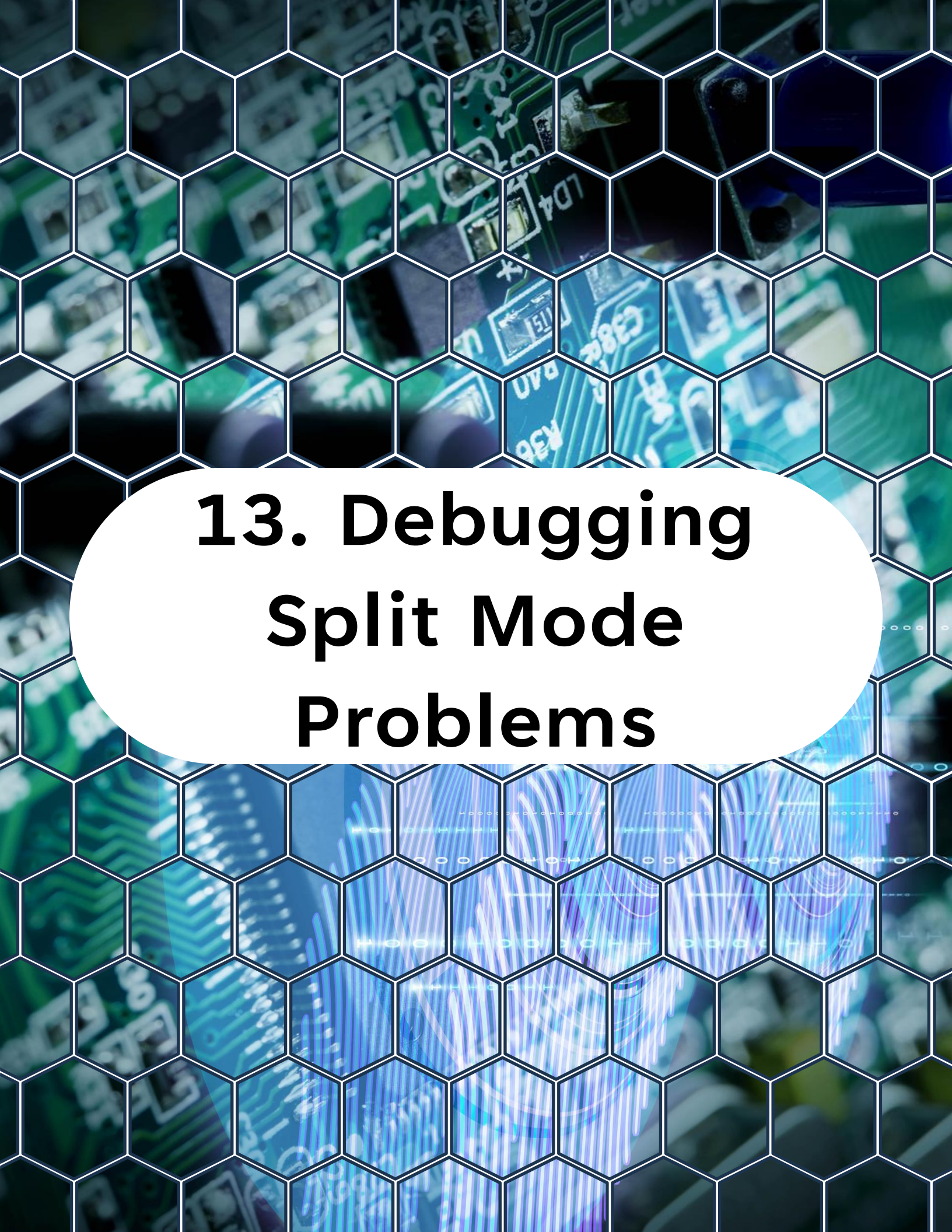
12. Minimal main() Example

12. Minimal main() Example

This ramps one duty cycle so you can see it change on the scope or LED.

```
1  /**
2   * Application entry point.
3   */
4  int main(void)
5  {
6      // Initializes TCA0 in Split mode.
7      tca0_split_pwm_init();
8
9      while (1)
10     {
11         for (uint8_t d = 0; d <= 100; d += 5)
12         {
13             tca0_split_set_lcmp0_duty_percent(d);
14             _delay_ms(50); // crude delay
15         }
16
17         for (int8_t d = 100; d >= 0; d -= 5)
18         {
19             tca0_split_set_lcmp0_duty_percent((uint8_t)d);
20             _delay_ms(50); // crude delay
21         }
22     }
23 }
```

Again, delay is used only for visibility. Later you will replace this with timer-driven modulation.



13. Debugging Split Mode Problems

13. Debugging Split Mode Problems

If you see no PWM:

- Confirm Split mode enabled: **CTRLD SPLITM** bit
- Confirm timer enabled: **CTRLA ENABLE** bit
- Confirm **WO** outputs enabled in **CTRLB (LCMPxEN/HCMPxEN)**
- Confirm pins are set as outputs
- Confirm **PORTMUX** routing matches pins you probed

If frequency is wrong:

- Confirm prescaler
- Confirm **LPER/HPER** values
- Confirm system clock (Day 3)

13. Debugging Split Mode Problems

If duty does not change:

- Confirm you are writing the correct **LCMPx** or **HCMPx**
- Confirm the corresponding output enable bit is set



14. What You Learned Today

14. What You Learned Today

By the end of Day 8, you now understand:

- The difference between **TCA** Single mode and Split mode
- How Split mode creates two independent 8-bit timers
- How to set **LPER/HPER** for two PWM frequencies
- How to set **LCMPx/HCMPx** for duty control
- How to enable multiple PWM outputs from one timer peripheral

This is a practical technique you will reuse in real products.



15. What Comes Next

15. What Comes Next

Day 9 - TCB Timers (Periodic Interrupts and One-Shot Timing)

Next, you will:

- Use **TCB** for periodic interrupts and timeouts
- Build a clean system tick without delay loops
- Create one-shot timeouts for state machines and comms
- Start building a reusable timing layer for the rest of the course

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>